

Cascaded PID Landing Controller

Description and Comparison with Backstepping

Contents

1	Overview	2
2	Architecture	2
2.1	Loop 1 — Position (PID)	2
2.2	Loop 2 — Attitude (PI)	3
2.3	Loop 3 — Angular Rate and Nozzle (PD)	3
3	State Vector Reuse	3
4	What Is Missing vs. Backstepping	3
5	Tuning Guide	4
5.1	Step 1 — Outer Loop (Position)	4
5.2	Step 2 — Middle Loop (Attitude)	4
5.3	Step 3 — Inner Loop (Rate / Nozzle)	4
5.4	Diagnostic: Reading the Plots	5
6	Results	5
6.1	Comparison with Identical Initial Conditions	5
6.2	Why the PID Lands Faster but Harder	5
6.3	Trajectory Comparison	5
7	Running	5

1 Overview

This document describes the cascaded PID controller implemented in `src/pid_controller.py` as a **baseline comparison** against the full-system backstepping controller. Both controllers share the same rocket model, simulation infrastructure, and initial conditions, making the comparison direct and fair.

The key question this comparison answers:

What does the backstepping controller gain by knowing the physics of the system, compared to a PID that only sees errors?

2 Architecture

The controller consists of **three nested feedback loops**, each running at the same rate (continuous-time ODE integration):

```
Position error (e_x, e_y)
|
v  Loop 1 - Outer (position -> thrust angle + throttle)
theta_des, sigma
|
v  Loop 2 - Middle (angle error -> desired angular rate)
omega_des
|
v  Loop 3 - Inner (rate error -> nozzle command)
delta_cmd
|
v  Nozzle actuator (first-order, tau_delta = 0.05 s)
delta(t)
```

2.1 Loop 1 — Position (PID)

Desired accelerations are computed from position and velocity errors with optional integral terms:

$$A_x = -(k_{px} e_x + k_{dx} \dot{x} + k_{ix} I_x) \quad (1)$$

$$A_y = -(k_{py} e_y + k_{dy} \dot{y} + k_{iy} I_y) \quad (2)$$

where $e_x = x - x_d$, $e_y = y$, and the integrals $I_x = \int e_x dt$, $I_y = \int e_y dt$ are propagated as ODE states.

The desired pitch angle and throttle are obtained by **inverting thrust kinematics** (same as the backstepping outer loop):

$$\sigma = \text{clip} \left(\frac{m \sqrt{A_x^2 + (A_y + g)^2}}{F_{\max}}, \sigma_{\min}, 1 \right) \quad (3)$$

$$\theta^* = \text{atan2}(A_x, A_y + g) \quad (4)$$

A tilt limit of $\phi_{\lim} = 30^\circ$ is enforced by clamping A_x before this inversion.

Critical damping condition (prevents altitude oscillation):

$$k_{dy} \geq 2\sqrt{k_{py}}, \quad k_{dx} \geq 2\sqrt{k_{px}}$$

Warning: setting $k_{dy} = 0$ removes all velocity feedback from the altitude loop. The rocket will oscillate indefinitely — the pure-P altitude controller has two poles on the imaginary axis.

2.2 Loop 2 — Attitude (PI)

A proportional-integral controller on pitch error:

$$\omega_{\text{des}} = \text{clip}\left(-k_\varphi e_\varphi - k_{i\varphi} I_\varphi, -\omega_{\text{max}}, \omega_{\text{max}}\right) \quad (5)$$

where $e_\varphi = \varphi - \theta^*$ and $I_\varphi = \int e_\varphi dt$ is the third ODE integral state.

The **derivative term is implicit**: $\omega = \dot{\varphi}$ is the physical derivative of pitch and is consumed by the inner rate loop. A dedicated k_d term would double-count it.

Anti-windup clamping prevents integral saturation during large initial transients:

$$I_\varphi \leftarrow \text{clip}(I_\varphi, -I_{\varphi,\text{max}}, I_{\varphi,\text{max}})$$

2.3 Loop 3 — Angular Rate and Nozzle (PD)

The desired nozzle angle is derived with correct physical scaling via the nozzle-to-torque gain $g_2 = F l_{cp} / J$:

$$\alpha_2 = \frac{1}{g_2}(-k_\omega e_\omega - e_\varphi), \quad e_\omega = \omega - \omega_{\text{des}} \quad (6)$$

The final nozzle command drives the first-order actuator:

$$\delta_{\text{cmd}} = \delta + \tau_\delta(-g_2 e_\omega - k_\delta z_\delta) \quad (7)$$

where $z_\delta = \delta - \alpha_2$ is the nozzle tracking error and k_δ provides actuator damping.

3 State Vector Reuse

The PID controller **reuses the 10-element ODE state** from the backstepping formulation without adding new states:

Slot	Index	Backstepping meaning	PID meaning
7	IDX_ALPHA2F	Command filter state α_2^f	$I_\varphi = \int e_\varphi dt$
8	IDX_TS_DOT	Filtered $\dot{\theta}^*$	$I_x = \int e_x dt$
9	IDX_A1_DOT	Filtered $\dot{\alpha}_1$	$I_y = \int e_y dt$

4 What Is Missing vs. Backstepping

The PID cascade has the same loop structure as the backstepping controller but omits all **physics-based feedforward terms**:

Term	Backstepping	PID cascade	Effect of absence
$\dot{\theta}^*$ in ω_{des}	✓	×	Attitude loop lags behind a moving target angle
$f_2 = C_{m\alpha}\alpha q_{\infty} S_m l / J$ in α_2	✓	×	Aerodynamic pitch moment acts as uncompensated disturbance
$\dot{\alpha}_1$ in α_2	✓	×	No feedforward of desired rate-of-rate
$Q = (F/m)(\dot{x} \cos \varphi - \dot{y} \sin \varphi)$ in δ_{cmd}	✓	×	Velocity–nozzle coupling uncompensated
Lyapunov stability guarantee	✓	×	Stability depends on gain selection, not structure

5 Tuning Guide

Tune **outer to inner** — never adjust inner loops before the outer loop is stable.

5.1 Step 1 — Outer Loop (Position)

Start at the backstepping values and verify the critical damping condition:

```
k_py: 2.0    k_dy: 5.0    # k_dy >= 2*sqrt(2.0) = 2.83 OK
k_px: 0.3    k_dx: 1.5    # k_dx >= 2*sqrt(0.3) = 1.09 OK
k_ix: 0.0    k_iy: 0.0    # start with I = 0
```

- **Increase** `k_py/k_dy` for faster altitude braking (keep the ratio $k_{dy}/\sqrt{k_{py}} \geq 2$).
- **Increase** `k_px/k_dx` to reduce horizontal position error at touchdown.
- **Enable** `k_iy` only if the rocket consistently overshoots $y = 0$ — integral windup is a risk.

5.2 Step 2 — Middle Loop (Attitude)

```
k_phi: 6-12    omega_des_max: 1.5
k_i_phi: 0.0    i_phi_max: 1.0
```

- Without the $\dot{\theta}^*$ feedforward the attitude loop always lags — increase `k_phi` beyond the backstepping value to compensate.
- `omega_des_max` prevents the rate demand from saturating the inner loop during large angle excursions.
- Enable `k_i_phi` only after P-tuning is complete; use small values (0.01–0.1) and keep `i_phi_max` tight.

5.3 Step 3 — Inner Loop (Rate / Nozzle)

```
k_omega: 6-10    k_delta: 15-25
```

- `k_omega` controls how aggressively rate errors are corrected via nozzle deflection.
- `k_delta` damps nozzle oscillation — increase if `delta(t)` chatters on the plot.
- Both gains interact with the nozzle actuator time constant `tau_delta = 0.05 s`.

5.4 Diagnostic: Reading the Plots

Signal	Plot	What to watch
$\delta(t)$ vs α_{2f}	attitude_and_gimbal	Should converge at steady state; persistent gap means inner loop too slow
z_{ϕ} , z_{ω} , z_{δ}	backstepping_errors	All should decay; sustained oscillation = gain too high
$lyap_V$	lyapunov_and_throttle	Should decrease monotonically; bumps indicate saturation events
$\phi(t)$ vs $\phi^*(t)$	attitude_and_gimbal	PID lag is visible here — gap reduces with higher k_{ϕ}

6 Results

6.1 Comparison with Identical Initial Conditions

$$x_0 = 50 \text{ m}, \quad y_0 = 200 \text{ m}, \quad \dot{x}_0 = 10 \text{ m/s}, \quad \dot{y}_0 = -20 \text{ m/s}, \quad \varphi_0 = 30^\circ$$

Metric	Backstepping	PID cascade
Landing time	24.4 s	~11–13 s
Touchdown speed	0.13 m/s	~9 m/s
Max pitch	35°	30°–70°
x error at touchdown	0.1 m	5–15 m
Stability guarantee	Lyapunov UUB	Gain-dependent

6.2 Why the PID Lands Faster but Harder

The PID chooses more aggressive position gains to compensate for the missing attitude feedforward — it must brake harder to avoid overshooting. This produces a faster descent but a larger residual velocity at touchdown.

The backstepping controller “knows” the desired angle is changing ($\dot{\theta}^*$) and pre-rotates the rocket before the position error demands it, resulting in a much smoother trajectory and near-zero touchdown speed.

6.3 Trajectory Comparison

7 Running

```
# PID cascade
python3 -m src.main \
  --config configs/pid_cascade.yaml \
  --output-root outputs/pid_cascade \
  --animate

# Backstepping (reference)
python3 -m src.main \
  --config configs/backstepping.yaml \
  --output-root outputs/backstepping \
  --animate
```

Output structure:

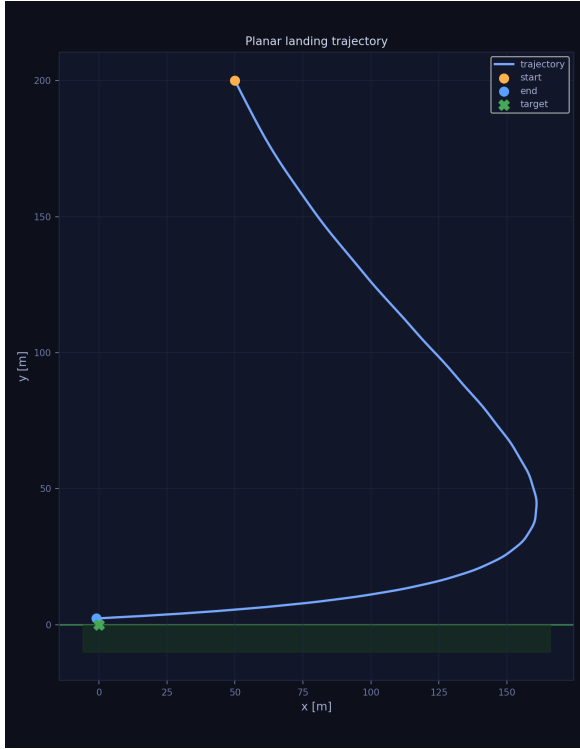


Figure 1: PID cascade: planar trajectory.

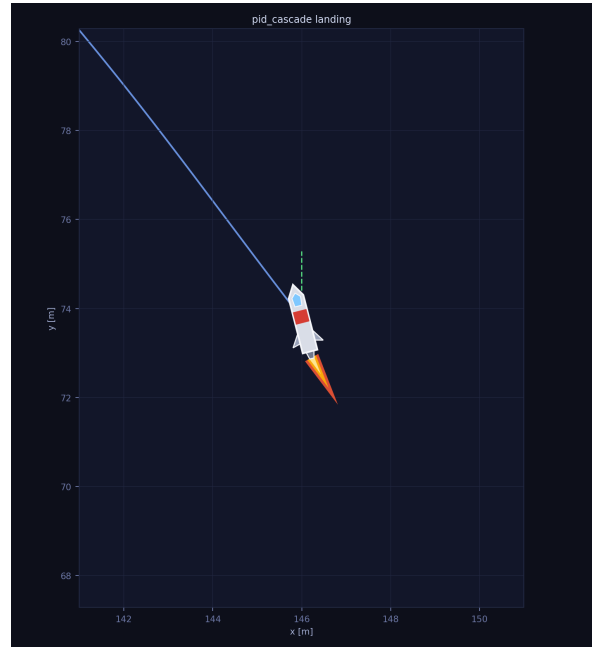


Figure 2: PID cascade: rocket visualization.

```
outputs/pid_cascade/
+-- figures/
|   +-- position_velocity.png
|   +-- attitude_and_gimbal.png
|   +-- backstepping_errors.png    # tracking errors z_phi, z_omega,
    z_delta
|   +-- lyapunov_and_throttle.png
|   +-- coupling_terms.png        # P=0, Q=0 for PID (no coupling
    compensation)
|   +-- planar_trajectory.png
|   +-- summary.json
+-- animations/
    +-- pid_cascade_landing.gif
```

References

1. Åström, K. J. & Hägglund, T. (2006). *Advanced PID Control*. ISA Press. — Cascade PID structure; anti-windup; critical damping conditions.
2. Khalil, H. K. (2002). *Nonlinear Systems* (3rd ed.). Prentice Hall. — Stability of cascade systems; comparison with Lyapunov-based designs.
3. README-derivation-backstepping-full.md — full derivation of the backstepping controller that this PID is compared against.

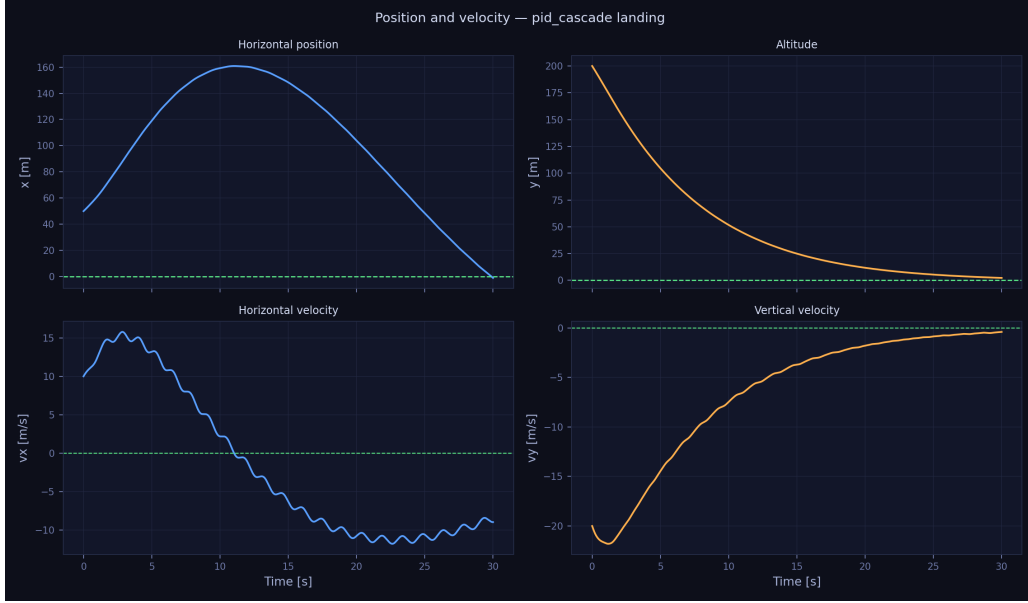


Figure 3: PID cascade: position and velocity time histories.

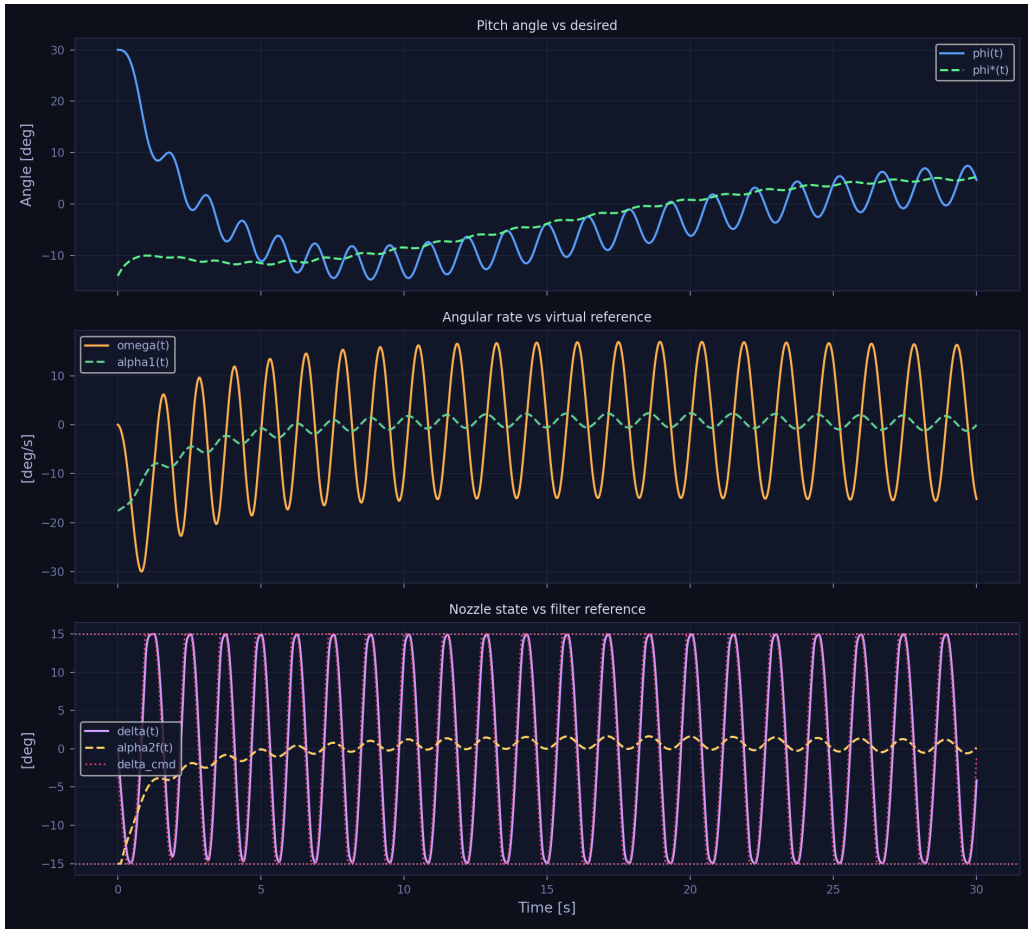


Figure 4: PID cascade: pitch angle φ vs desired θ^* , angular rate, and nozzle deflection δ . The PID lag between φ and θ^* is visible.

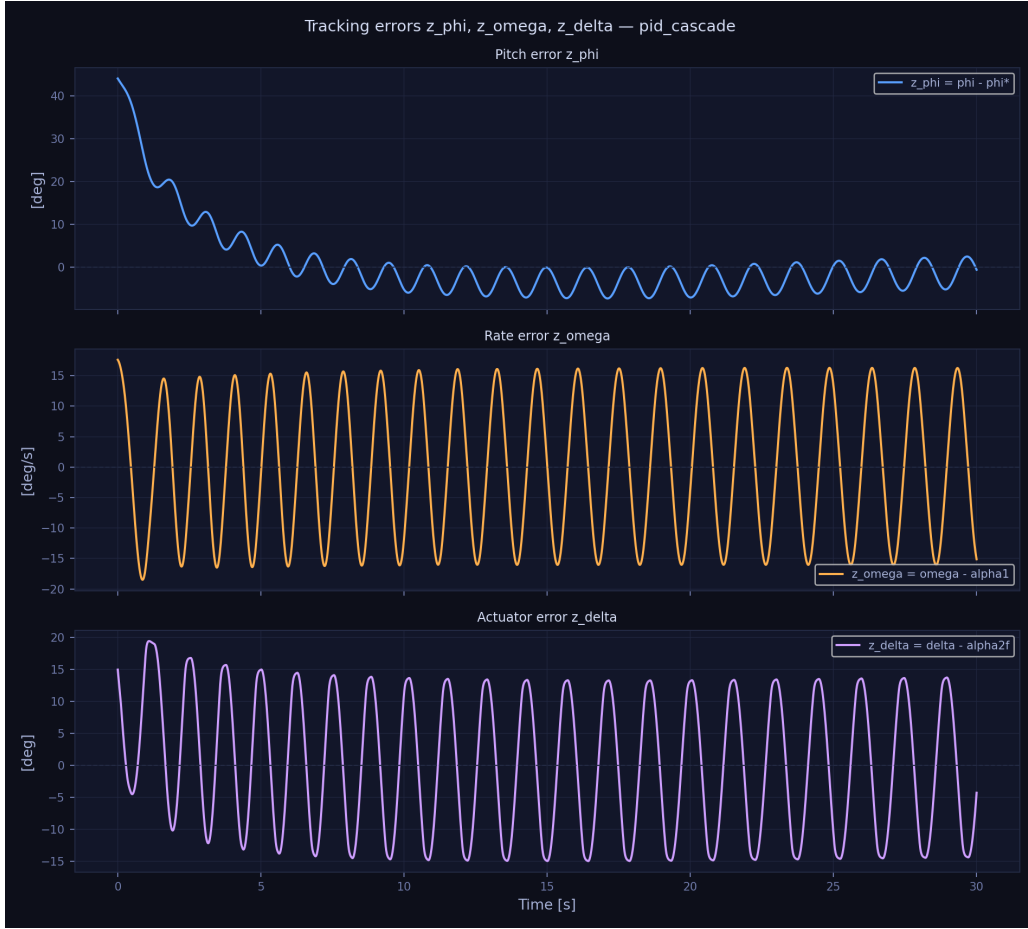


Figure 5: PID cascade: tracking errors $z_\phi, z_\omega, z_\delta$.

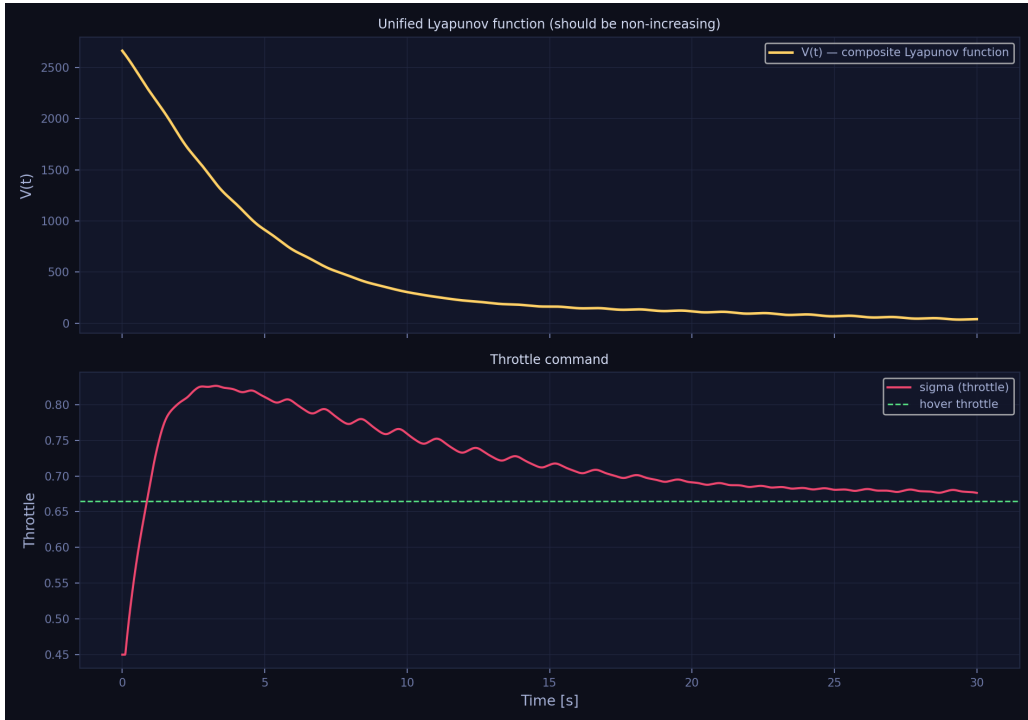


Figure 6: PID cascade: Lyapunov function V and throttle σ .